

Defending Against Knowledge Poisoning Attacks During Retrieval-Augmented Generation

Kennedy Edemacu*, Vinay M. Shashidhar†, Dan Abudu‡, Micheal Tuape§, Beakcheol Jang¶, Jong Wook Kim||

Abstract—Retrieval-Augmented Generation (RAG) has emerged as a powerful approach to boost the capabilities of large language models (LLMs) by incorporating external, up-to-date knowledge sources. However, this introduces a potential vulnerability to knowledge poisoning attacks, where attackers can compromise the knowledge source to mislead the generation model. For example, a class of such attacks injects adversarial texts that steer the model to generate an attacker-chosen response to a target question. In this work, we propose novel defense methods, FilterRAG and ML-FilterRAG, to mitigate such attacks. First, we propose a new property to differentiate between adversarial and clean texts in the knowledge data source. Next, we employ this property to filter out adversarial texts from clean ones in the design of our proposed approaches. Evaluation of these methods using benchmark datasets demonstrates their effectiveness, with performances close to those of the original RAG systems.

Index Terms—Large language models, retrieval-augmented generation, and knowledge poisoning attack

I. INTRODUCTION

RAG systems have emerged as a powerful paradigm for enhancing LLMs with external knowledge [1]–[4]. By retrieving relevant documents at inference time, RAG improves factual accuracy and enables domain-specific reasoning, addressing the limitations of LLMs in incorporating up-to-date knowledge [5], [6]. However, this reliance on external knowledge sources also introduces a critical vulnerability: *knowledge poisoning attacks*, where adversarial content injected into the retrieval corpus can mislead the model into generating incorrect or harmful responses [6]–[8].

Recent work has shown that RAGs are susceptible to such attacks, as the generator implicitly trusts retrieved content without verifying its integrity [6]. In adversarial settings, poisoned documents can be included in the top retrieved results, significantly degrading model performance. Existing defenses largely focus on analyzing internal model behavior or using ensemble-based strategies [5], [9], but they fail to address a key gap: the absence of a principled mechanism to distinguish between adversarial and clean documents *before* they influence generation in both black-box and white-box settings.

In this work, we propose a simple yet effective defense strategy that introduces a filtering step between retrieval and generation. Our approach is based on a novel statistical signal,

termed *Freq-Density*, which captures the distributional characteristics of retrieved texts. Intuitively, adversarial documents exhibit distinct frequency-density patterns compared to clean documents, enabling their identification without requiring access to model internals. Building on this insight, we develop two methods: *FilterRAG*, a lightweight threshold-based filtering approach, and *ML-FilterRAG*, which leverages multiple features within a supervised classification framework.

We evaluate our methods under both black-box and white-box poisoning attacks across multiple datasets, including HotpotQA, MS-MARCO, and Natural Questions. We primarily evaluate against a non-adaptive poisoning attack in which the adversary does not optimize poisoning content specifically against the proposed detector. Experimental results demonstrate that our approaches significantly reduce the impact of adversarial documents while maintaining competitive performance on clean data.

Our main contributions are as follows:

- We introduce *Freq-Density*, a novel statistical property for distinguishing adversarial and clean retrieved texts.
- We propose *FilterRAG* and *ML-FilterRAG*, two effective filtering-based defenses for RAG systems.
- We demonstrate robustness against both black-box and white-box knowledge poisoning attacks.
- We show that our methods preserve downstream task performance while mitigating adversarial influence.

II. PRELIMINARIES

A. Threat Model

We maintain the same threat model as [6].

Attacker’s Goal: We assume that the attacker pre-selects a set of target questions, denoted as $\mathcal{Q} = \{q_1, q_2, \dots, q_m\}$, and a corresponding set of desired responses $\mathcal{R} = \{r_1, r_2, \dots, r_m\}$. The attacker’s goal is to subvert the RAG system so that for each target question $q_i \in \mathcal{Q}$, the system generates the attacker-desired response $r_i \in \mathcal{R}$. For example, if the target question is, *which disease is caused by the human immunodeficiency virus?* the compromised RAG system would erroneously answer *Syphilis* instead of the correct response *AIDS*.

Attacker’s Capabilities: For each target question q_i , we assume that the attacker can inject a set of n poisoned texts denoted as $\mathcal{P} = \{p_i^1, p_i^2, \dots, p_i^n\}$ directly into the knowledge database $\mathcal{D}$. We then consider both black-box and white-box settings of the RAG retriever. In the black-box setting, we assume that the attacker has no access to the retriever’s parameters and cannot directly query it. However, in the white-box setting, the attacker is assumed to have access to the retriever’s parameters.

*The City University of New York-College of Staten Island, US. (Email: kennedy.edemacu@csi.cuny.edu). †Northern Michigan University, US. ‡Aston University, UK. §Lappeenranta-Lahti University of Technology, Finland. ¶Yonsei University, South Korea. ||Sangmyung University, South Korea.

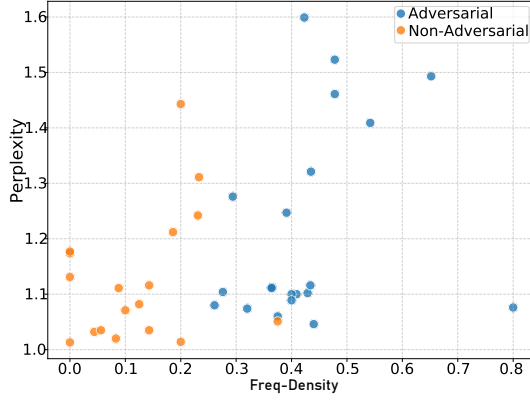


Fig. 1: Pair Plot for Freq-Density vs Perplexity

B. The Knowledge Corruption Attack

We consider the knowledge corruption attack in [6], which is a constrained problem in which a set of malicious texts $\tilde{\mathcal{D}} = \{p_i^j | i = 1, 2, \dots, m, j = 1, 2, \dots, n\}$ is constructed such that the LLM in the RAG system generates the attacker-desired response $r_i \in \mathcal{R}$ for the target question $q_i \in \mathcal{Q}$ when utilizing k texts retrieved from the compromised knowledge database $\mathcal{D} \cup \tilde{\mathcal{D}}$. Formally, the attack is represented as:

$$\max_{\tilde{\mathcal{D}}} \frac{1}{m} \sum_{i=1}^m \mathbb{I} \left(\text{LLM} \left(q_i; \mathcal{E}(q_i; \mathcal{D} \cup \tilde{\mathcal{D}}) \right) = r_i \right) \quad (1)$$

where; $\mathbb{I}(\cdot)$ is an indicator function with value 1 if the condition is satisfied and 0 otherwise. $\mathcal{E}(q_i; \mathcal{D} \cup \tilde{\mathcal{D}})$ is the k retrieved texts from the compromised knowledge database $\mathcal{D} \cup \tilde{\mathcal{D}}$.

C. Motivation

RAG systems implicitly trust retrieved documents, making them vulnerable to poisoning attacks where a few adversarial documents can significantly influence generation. In particular, an attack succeeds when a malicious text $\mathcal{P}$ appears in the top- k retrieved texts for a target query q_i , i.e., $\mathcal{P} \in \mathcal{E}(q_i; \mathcal{D} \cup \tilde{\mathcal{D}})$.

A key challenge is that adversarial texts are optimized to resemble relevant documents under standard retrieval signals, allowing them to be ranked among top results and included in the retrieved set [10].

This motivates an intermediate filtering step over the retrieved documents prior to generation. While adversarial texts are difficult to detect using retrieval-based similarity, we observe that they exhibit distinct token frequency and distribution patterns compared to clean documents. This insight forms the basis of our *Freq-Density* measure for identifying and filtering adversarial content. When combined with other features, it can also provide useful insights into the structure of the retrieved text. For example, Figure 1 visualizes Freq-Density against perplexity, where distinct clusters of clean and adversarial data points can be observed, supporting our hypothesis.

III. METHODOLOGY

A. Overview

We introduce a filtering step in RAG systems that operates between retrieval and generation to remove potentially adver-

sarial documents from the top- k retrieved set. Our approach is based on a statistical signal, *Freq-Density*, which captures differences between clean and adversarial texts. These differences are used to filter out the adversarial texts from the retrieved documents. Figure 2 presents a high-level illustration of our framework. We develop two variants: *FilterRAG*, a threshold-based method, and *ML-FilterRAG*, a supervised approach for improved detection.

B. Filtration of Adversarial Texts

This section presents our proposed frameworks for filtering out adversarial texts. The frameworks aim to minimize the influence/presence of adversarial texts within the context used during the generation phase.

Problem Statement: Given a set of target queries $\mathcal{Q} = \{q_1, q_2, \dots, q_m\}$, a corresponding set of desired responses $\mathcal{R} = \{r_1, r_2, \dots, r_m\}$ and a poisoned knowledge database $\mathcal{D} \cup \tilde{\mathcal{D}}$, we seek to minimize Eq. 1 so that the target query $q_i \in \mathcal{Q}$ does not result in its corresponding desired response $r_i \in \mathcal{R}$. Formally represented as:

$$\min_{\tilde{\mathcal{D}}} \frac{1}{m} \sum_{i=1}^m \mathbb{I} \left(\text{LLM} \left(q_i; \mathcal{E}(q_i; \mathcal{D} \cup \tilde{\mathcal{D}}) \right) = r_i \right) \quad (2)$$

subject to $\|\tilde{\mathcal{D}}\| \rightarrow 0$. By incorporating the latter constraint, we reduce the chances of obtaining the desired response r_i for the target query q_i . Our proposed methods are:

1) *FilterRAG (Threshold-Based Approach)*: This is a two-stage approach based on selecting a threshold value for our statistical property.

Stage 1 - Statistical Property Extraction: Given a query set $\mathcal{Q} = \{q_1, q_2, \dots, q_m\}$ and a poisoned knowledge database $\mathcal{D} \cup \tilde{\mathcal{D}}$, we leverage a smaller language model (SLM) to extract the *Freq-Density* statistical property for each retrieved item. In particular, during the retrieval phase, for each query $q_i \in \mathcal{Q}$, we retrieve the top- s items from the poisoned knowledge base $\mathcal{D} \cup \tilde{\mathcal{D}}$ rather than the traditional top- k items. Then, for each of the top- s retrieved items d_j in the top- s , we input both the query q_i and d_j into SLM to generate an output a_j .

$$a_j = \text{SLM}(q_i, d_j) \quad \text{for } j \in \{1, 2, \dots, s\} \quad (3)$$

Next, we define our *Freq-Density* as follows:

$$\text{Freq-Density} = \frac{\sum_{w \in (q_i \oplus a_j) \cap d_j} \text{Freq}(w, d_j)}{\text{UniqueWords}(d_j)} \quad (4)$$

where, $\oplus$ means concatenation, $(q_i \oplus a_j) \cap d_j$ are semantically similar words common to $(q_i \oplus a_j)$ and d_j , i.e., word pairs in $(q_i \oplus a_j)$ and d_j whose computed similarities exceed a predetermined threshold, $\text{Freq}(w, d_j)$ denotes the frequency of the word w within the text d_j and $\text{UniqueWords}(d_j)$ denotes the total number of unique words in d_j . The rationale behind this metric is rooted in the attackers' strategy. To both steer the LLM to their desired response r_i for the target query q_i , and to ensure high retrieval relevance, attackers populate their adversarial texts with words that are semantically similar to the target queries and their corresponding desired responses. This overlap makes *Freq-Density* a valuable indicator for

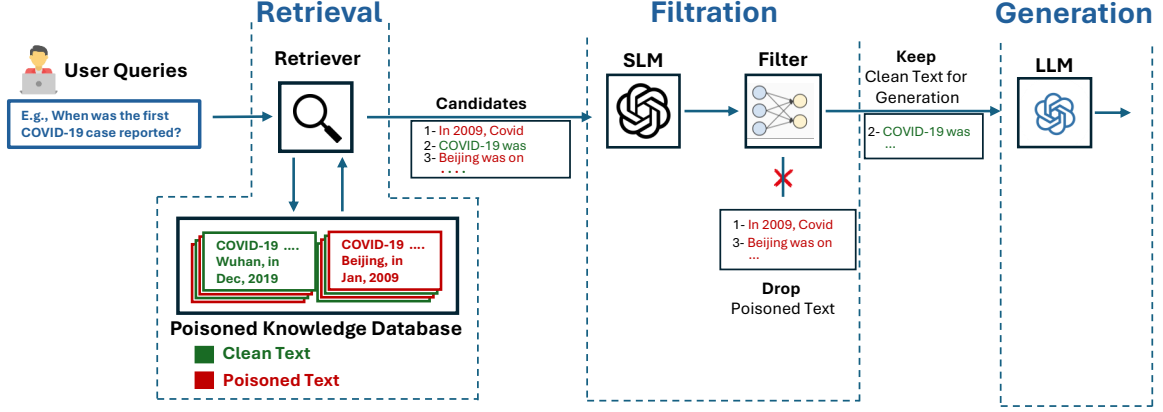


Fig. 2: A high-level illustration of our framework. A filtration phase is integrated into the RAG to filter out adversarial texts before the generation phase. A smaller language model (SLM) generates an intermediate response, which is input together with the query and retrieved text into a filter function to determine if the retrieved text is adversarial or not.

identifying malicious texts. Figure 1 presents an empirical illustration of this claim.

Stage 2 - Removal of Adversarial Texts: We then employ the defined *Freq-Density* to identify adversarial texts for removal by setting a threshold ϵ . We define a filter indicator function (Eq. 5) to determine whether a retrieved text d_j is adversarial or not.

$$\text{Filter}(d_j) = \begin{cases} 1, & \text{if Freq-Density} < \epsilon \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

We feed each item in the top- s through the filter function. Next, we remove the items whose return value from the filter function is 0. From the remaining items, the top- k items which constitute the context used to prompt the LLM are chosen. The effectiveness of this method depends on the choice of ϵ . This parameter is crucial for balancing the trade-off between the overall performance and the presence of adversarial texts in the final top- k items.

2) *ML-FilterRAG (Machine Learning Approach)*: The effectiveness of our first approach hinges on the choice of ϵ . A high value of ϵ risks allowing adversarial text samples to appear in the final top- k items. Meanwhile, a low value could filter out legitimately clean texts. Striking the right balance is crucial and yet challenging. Inspired by the work in [6], which used perplexity to attempt to filter out adversarial texts, we believe that combining multiple features can effectively address the challenge of ϵ selection. Thus, we propose ML-FilterRAG, a machine learning-based filtering approach. In this method, a filtering machine learning model takes multiple features extracted from each retrieved text d_j as input and predicts whether d_j is an adversarial sample. Similarly, this is a two-stage method.

Stage 1 - Feature Extraction: Similar to the extraction method discussed in III-B1, we employ an SLM that takes the target query q_i and a retrieved text d_j as input and subsequently outputs a_j . However, on top of the *Freq-Density*, we also compute perplexity, joint log probability of the SLM’s output a_j and sum of frequencies of semantically similar

words between $(q_i \oplus a_j)$ and d_j . We incorporate these as supplementary features for our filter model.

Stage 2 - Adversarial Text Prediction: Given a light-weight trained machine learning model $\mathcal{M}$, each of the retrieved items in the top- s undergoes the feature extraction process discussed previously. Once these feature values are extracted, they are fed into $\mathcal{M}$ (according to Eq. 6) to predict whether the item d_j is an adversarial text or not.

$$\text{pred}_j = \mathcal{M}(\text{Feature}(q_i, d_j)) \quad (6)$$

Where, pred_j can be adversarial or non-adversarial, and $\text{Feature}(\cdot)$ is a feature extraction function. Finally, the top- k items are chosen from the non-adversarial items as context to prompt the LLM. By generating features that capture semantic similarities, we believe that our approaches provide a broader capability for identifying adversarial texts. To train $\mathcal{M}$, we use the supervised learning approach in which the labeled datasets of retrieved texts for each target query are annotated as adversarial and non-adversarial. For each training instance, we extract features using the method discussed above. These features capture the statistical cues indicative of adversarial texts. $\mathcal{M}$ is trained to learn the mappings between these features and their corresponding labels. Once trained, $\mathcal{M}$ serves as an effective filter that predicts if d_j is adversarial based on its computed features.

IV. EXPERIMENTAL SETUP

Datasets. We evaluate our methods on three benchmark QA datasets: *MS-MARCO* [11], *Natural Questions (NQ)* [12], and *HotpotQA* [13]. We adopt the poisoned versions from [6], where each dataset contains 100 target questions, each paired with ground-truth and attacker-desired answers, along with 5 injected adversarial texts per query.

For ML-FilterRAG, we augment this setup by generating 5 additional adversarial texts per query using GPT-4o, ensuring that they produce attacker-desired outputs and rank within the top-15 retrieved results. We use a balanced set of adversarial and clean texts for training and reserve the remaining data for evaluation. For white-box attacks, we adopt HotFlip [6].

RAG Configuration. We use Contriever [14] with dot-product similarity [1], [6]. For filtering, we employ LLaMA-3 [15] as an SLM. FilterRAG uses a default threshold $\epsilon = 0.2$. This value was empirically determined for the datasets employed in this study and is guaranteed to be optimal for all RAG settings and KBs. ML-FilterRAG employs dataset-specific models: XGBoost (NQ) and Random Forest (HotpotQA, MS-MARCO). Semantic similarity is computed using a sentence transformer with cosine threshold 0.6.

For generation, we use GPT-3.5 [16], GPT-4 [17], GPT-4o [18], and LLaMA-3, with temperature set to 0.1 and prompts from [6].

Metrics. We evaluate performance using: (i) *Adversarial Text Ratio (ATR)*, measuring the fraction of adversarial texts in top- k retrieval; (ii) *Attack Success Rate (ASR)* [6], measuring the fraction of attacker-desired outputs; and (iii) *Accuracy*, measuring alignment with ground-truth answers.

Baselines. We compare against PoisonedRAG [6] and CleanRAG (no adversarial texts), assessing robustness and performance preservation.

Evaluation Protocol. We use a controlled setup where the top- $s = 4$ retrieved candidates contain a balanced mix of adversarial and clean texts, from which top- $k = 2$ are selected for generation. We set $m = 10$ and repeat experiments 10 times to ensure coverage across all target queries.

V. EXPERIMENTAL RESULTS

A. Main Results

We evaluate our methods on HotpotQA, MS-MARCO, and NQ under both black-box and white-box PoisonedRAG attacks. The main results for the black-box setting are shown in Table I.

Our methods, *FilterRAG* and *ML-FilterRAG*, consistently improve robustness in the presence of adversarial texts. For instance, with GPT-3.5, FilterRAG achieves accuracies of 88.1%, 82.0%, and 75.6% on HotpotQA, MS-MARCO, and NQ, respectively, while ML-FilterRAG further improves performance to 90.3%, 85.1%, and 81.1%. At the same time, both methods achieve substantially lower ASR and ATR compared to the PoisonedRAG baseline. For example, FilterRAG reduces ASR to 8.2%, 9.0%, and 3.0%, and ML-FilterRAG achieves 9.0%, 6.0%, and 2.0% across the three datasets. Similarly, ATR values are reduced to near-zero levels, indicating effective removal of adversarial texts. Similar patterns are also observed in the white-box setting shown in Table II.

These results demonstrate that reducing the presence of adversarial texts (low ATR) directly leads to lower ASR, thereby limiting the influence of poisoned content on generation. This behavior is consistent with our design, where filtering is performed prior to generation based on statistical properties of retrieved texts.

We further observe that *ML-FilterRAG* consistently outperforms *FilterRAG* in terms of accuracy. This is because FilterRAG relies on a single threshold over Freq-Density, which may occasionally remove clean texts, whereas ML-FilterRAG leverages multiple features to better distinguish adversarial and legitimate content.

Performance also varies across datasets. Datasets with longer passages (e.g., MS-MARCO) provide richer contextual signals, making adversarial patterns easier to detect, while shorter or more concise datasets present a more challenging setting. Nevertheless, our methods consistently achieve lower ATR and ASR while maintaining accuracy close to the CleanRAG baseline.

Finally, while our methods do not surpass CleanRAG in accuracy due to occasional misclassification of clean texts, they significantly improve robustness over PoisonedRAG. In particular, ML-FilterRAG mitigates this trade-off more effectively, whereas FilterRAG is sensitive to the choice of ϵ , reflecting a balance between filtering strength and accuracy. Furthermore, although the proposed methods substantially reduce attack success for non-adaptive attacks, an adaptive adversary that optimizes poisoning embeddings to remain below the detection threshold may partially evade the defense.

B. Ablation Studies

We also performed ablation studies and present the results as follows.

1) *Varying Similarity Threshold:* The similarity threshold determines the level of exactness between the query-answer combination and the context text words. This exactness is crucial for both the FilterRAG and ML-FilterRAG methods. As a result, we perform experiments that demonstrate how this exactness affects our proposed methods. We present the results in Table III. From the results, we observe that at the similarity threshold of 1.0, FilterRAG and ML-FilterRAG both have high ATR and ASR, and low accuracies. As the similarity threshold decreases from 0.9 to 0.6, both FilterRAG and ML-FilterRAG show decreasing ATR and ASR, while their accuracies increase. Beyond the similarity threshold value of 0.6, FilterRAG shows decreasing ATR, ASR and accuracy values, while ML-FilterRAG shows increasing ATR and ASR values, but decreasing accuracy values. This result demonstrates that, at highest similarity threshold of 1.0, only query-answer combination words that perfectly match the context text words are considered. This level of perfection can allow attackers to evade our proposed methods simply using modified word versions or synonyms and allow more adversarial texts to go undetected by our methods. A reason our proposed methods show high ATR and ASR values, and low accuracy at similarity threshold of 1.0. As we relax the level of exactness by reducing the similarity threshold, the match between query-answer combination and context text words is now measured at semantic level. This is reflected by the decrease in both ATR and ASR values and the increase in accuracy values for similarity thresholds of 0.9 to 0.6. However, further decrease in similarity threshold values beyond 0.6, blurs the boundary between legitimate and adversarial texts and results in poor performance for both FilterRAG and ML-FilterRAG.

2) *Effect of ϵ -value:* The performance of FilterRAG depends on the choice of ϵ . We demonstrate this through experiments. We present the experimental results for the MS-MARCO dataset in Table IV. From the result, we observe that at the lowest ϵ value of 0.1, FilterRAG achieves its

TABLE I: Performance Comparison of RAG Frameworks Across Different LLMs for Black-box Attack

Dataset	Framework	ATR ↓	GPT-3.5		GPT-4		GPT-4o		LLaMA-3	
			ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑
HotpotQA	CleanRAG	0.000	0.080	0.913	0.090	0.908	0.090	0.907	0.310	0.665
	PoisonedRAG	1.000	0.940	0.042	0.900	0.000	0.930	0.000	0.980	0.001
	FilterRAG ($\epsilon = 0.2$)	0.000	0.082	0.881	0.090	0.870	0.091	0.820	0.380	0.536
	ML-FilterRAG	0.015	0.090	0.903	0.091	0.905	0.094	0.899	0.330	0.541
MS-MARCO	CleanRAG	0.000	0.060	0.859	0.050	0.851	0.050	0.867	0.310	0.667
	PoisonedRAG	0.825	0.820	0.160	0.824	0.173	0.834	0.128	0.912	0.082
	FilterRAG ($\epsilon = 0.2$)	0.065	0.090	0.820	0.090	0.840	0.080	0.840	0.430	0.509
	ML-FilterRAG	0.045	0.060	0.851	0.060	0.849	0.080	0.831	0.400	0.565
NQ	CleanRAG	0.000	0.010	0.831	0.020	0.833	0.020	0.831	0.140	0.575
	PoisonedRAG	0.980	0.880	0.108	0.870	0.119	0.864	0.118	0.980	0.100
	FilterRAG ($\epsilon = 0.2$)	0.010	0.030	0.756	0.030	0.818	0.030	0.773	0.190	0.480
	ML-FilterRAG	0.030	0.020	0.811	0.030	0.810	0.030	0.803	0.180	0.544

TABLE II: Performance Comparison of RAG Frameworks Across Different LLMs for White-box Attack

Dataset	Framework	ATR ↓	GPT-3.5		GPT-4		GPT-4o		LLaMA-3	
			ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑	ASR ↓	Accuracy ↑
HotpotQA	CleanRAG	0.000	0.080	0.913	0.090	0.908	0.090	0.907	0.310	0.665
	PoisonedRAG	1.000	0.860	0.056	0.870	0.022	0.872	0.021	0.980	0.001
	FilterRAG ($\epsilon = 0.2$)	0.000	0.005	0.721	0.090	0.733	0.090	0.731	0.335	0.600
	ML-FilterRAG	0.030	0.090	0.729	0.090	0.745	0.080	0.755	0.330	0.650
MS-MARCO	CleanRAG	0.000	0.060	0.859	0.050	0.851	0.050	0.867	0.310	0.667
	PoisonedRAG	0.980	0.720	0.074	0.710	0.107	0.750	0.102	0.960	0.030
	FilterRAG ($\epsilon = 0.2$)	0.125	0.120	0.721	0.110	0.729	0.110	0.706	0.320	0.596
	ML-FilterRAG	0.155	0.110	0.762	0.110	0.777	0.112	0.782	0.318	0.629
NQ	CleanRAG	0.000	0.010	0.831	0.020	0.833	0.020	0.831	0.140	0.575
	PoisonedRAG	1.000	0.730	0.031	0.700	0.033	0.720	0.046	0.970	0.010
	FilterRAG ($\epsilon = 0.2$)	0.025	0.050	0.823	0.050	0.816	0.050	0.828	0.160	0.500
	ML-FilterRAG	0.040	0.060	0.824	0.050	0.827	0.050	0.829	0.170	0.542

TABLE III: Performances with Varying Similarity Threshold Values for MS-MARCO Dataset using GPT-4

Method		Similarity Threshold							
		1.0	0.9	0.8	0.7	0.6	0.5	0.4	
FilterRAG	ATR ↓	0.415	0.080	0.070	0.065	0.065	0.020	0.000	
	ASR ↓	0.260	0.080	0.090	0.090	0.090	0.070	0.050	
	Accuracy ↑	0.621	0.827	0.831	0.833	0.840	0.766	0.497	
ML-FilterRAG	ATR ↓	0.425	0.090	0.080	0.060	0.045	0.195	0.480	
	ASR ↓	0.290	0.090	0.086	0.070	0.060	0.120	0.230	
	Accuracy ↑	0.589	0.834	0.834	0.844	0.849	0.793	0.538	

TABLE IV: FilterRAG Performance with respect to ϵ -Values for MS-MARCO Dataset using GPT-4

		ϵ -value				
		0.1	0.2	0.3	0.4	0.5
ATR ↓	0.005	0.065	0.230	0.540	0.765	
ASR ↓	0.050	0.090	0.200	0.320	0.540	
Accuracy ↑	0.624	0.840	0.764	0.462	0.280	

lowest ATR and ASR, however the accuracy is low, at only 62.4%. As ϵ -value increases to 0.2, FilterRAG achieves the best performance at an accuracy of 84.0%. However, its ATR and ASR begin to drop. Further increase in ϵ -values from 0.2 results in more increase in ATR and ASR values with decreasing accuracy. At lower ϵ -values, the majority of adversarial texts from top- s are filtered out. However, some legitimate texts are mistakenly filtered out too. This is why the

ATR and ASR are lower, but the accuracy is relatively low as well. The accuracy begins to increase with the increasing ϵ -values because more legitimate texts are being captured in the top- k . But, this affects both ATR and ASR, as more adversarial texts sneak into the top- k . This becomes more evident at higher ϵ -values. This result emphasizes that choosing an ϵ -value is a trade-off between minimizing the number of adversarial texts in the retrieved top- s and achieving better performance.

C. Real-World Deployment Considerations

FilterRAG and ML-FilterRAG are designed as modular filtration layers between retrieval and generation, enabling seamless integration into existing RAG systems without modifying retrievers or language models. This filtration step introduces additional overhead due to SLM inference and similarity computations, particularly as the number of retrieved documents

increase. For example, in our experiment, the filtration introduces an additional average computation time of 17.37 and 17.02 seconds per query for ML-FilterRAG and FilterRAG, respectively. Potentially, this latency can be reduced through adjusting top- s or caching feature computations.

Our ablation studies show that tuning similarity thresholds and ϵ enables a trade-off between accuracy and efficiency. ML-FilterRAG provides a better balance, achieving higher accuracy with minimal impact on ATR and ASR. Overall, both methods offer practical and scalable defenses for real-world RAG deployments.

VI. RELATED WORK

RAG Knowledge Poisoning Attacks. Recent work has demonstrated the vulnerability of RAG systems to knowledge poisoning attacks. GARAG [19] corrupts the knowledge database through low-level perturbations. PRCAP [20] introduces adversarial texts by perturbing discrete tokens to increase similarity with target queries. Topic-fliprag [7] manipulates document content to influence query-specific outputs, while [8] proposes stealthy perturbations for RAG-based recommender systems to promote or demote items. These methods highlight how adversarial documents can be injected and retrieved to compromise RAG outputs.

Defense Against Knowledge Poisoning. Few defenses have been proposed to mitigate these attacks. Tan *et al.* [5] detect adversarial texts using LLM internal activations, requiring white-box access. Xiang *et al.* [9] use ensemble-based generation, which remains vulnerable when adversarial texts dominate retrieval. Zou *et al.* [6] propose paraphrasing, perplexity filtering, and duplicate removal, though these approaches are not sufficiently robust for reliably filtering adversarial content.

Our Position. In contrast, our FilterRAG and ML-FilterRAG operate in both black-box and white-box settings without relying on model internals or ensemble generation. Our methods instead leverage statistical properties of retrieved texts to identify and filter adversarial documents prior to generation, addressing the reliability of the retrieved context.

VII. CONCLUSION

We studied the vulnerability of retrieval-augmented generation systems to knowledge poisoning attacks and proposed a filtering-based defense framework. Our methods, *FilterRAG* and *ML-FilterRAG*, leverage statistical properties of retrieved texts to identify and remove adversarial documents prior to generation. Experimental results show that our approaches effectively mitigate adversarial influence while preserving performance on clean data.

A limitation of our work is that detection relies on statistical patterns, which may be less effective against highly adaptive adversaries. Future work includes designing more robust signals against highly adaptive adversaries, investigating how different features influence $\mathcal{M}$, and extending the filtering mechanisms to broader retrieval and multimodal settings.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. S. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *EMNLP (1)*, 2020, pp. 6769–6781.
- [3] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International conference on machine learning*. PMLR, 2022, pp. 2206–2240.
- [4] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du *et al.*, “Lamda: Language models for dialog applications,” *arXiv preprint arXiv:2201.08239*, 2022.
- [5] X. Tan, H. Luan, M. Luo, X. Sun, P. Chen, and J. Dai, “Knowledge database or poison base? detecting rag poisoning attack through llm activations,” *arXiv preprint arXiv:2411.18948*, 2024.
- [6] W. Zou, R. Geng, B. Wang, and J. Jia, “Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models,” *arXiv preprint arXiv:2402.07867*, 2024.
- [7] Y. Gong, Z. Chen, M. Chen, F. Yu, W. Lu, X. Wang, X. Liu, and J. Liu, “Topic-fliprag: Topic-orientated adversarial opinion manipulation attacks to retrieval-augmented generation models,” *arXiv preprint arXiv:2502.01386*, 2025.
- [8] F. Nazary, Y. Deldjoo, T. Di Noia, and E. Di Sciascio, “Stealthy llm-driven data poisoning attacks against embedding-based retrieval-augmented recommender systems,” in *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*, 2025, pp. 98–102.
- [9] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably robust rag against retrieval corruption,” *arXiv preprint arXiv:2405.15556*, 2024.
- [10] M. Mozes, P. Stenetorp, B. Kleinberg, and L. D. Griffin, “Frequency-guided word substitutions for detecting textual adversarial examples,” *arXiv preprint arXiv:2004.05887*, 2020.
- [11] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, “Ms marco: A human generated machine reading comprehension dataset,” *choice*, vol. 2640, p. 660, 2016.
- [12] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, “Natural questions: a benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.
- [13] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “Hotpotqa: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.
- [14] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised dense information retrieval with contrastive learning,” *arXiv preprint arXiv:2112.09118*, 2021.
- [15] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [17] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [18] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
- [19] S. Cho, S. Jeong, J. Seo, T. Hwang, and J. C. Park, “Typos that broke the rag’s back: Genetic attack on rag pipeline by simulating documents in the wild via low-level perturbations,” *arXiv preprint arXiv:2404.13948*, 2024.
- [20] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, “Poisoning retrieval corpora by injecting adversarial passages,” *arXiv preprint arXiv:2310.19156*, 2023.